



Introduction To Algorithms

Week 5 - Recursion, Iteration, and Strategy

"Same Problem, Different Strategy"

1



Terms to Remember

Term	Brief Meaning
Strategy	A general approach for solving a problem
Correctness	Whether the algorithm produces the expected result
Performance	How efficiently the algorithm uses work or resources
Scalability	How the approach behaves as size or demand grows
Elasticity	Ability to handle changing demand or scale when resources change
Polynomial	A growth pattern that can be expressed with powers such as n , n^2 , or n^3
NP-Hard	A class of problems that are at least as hard as the hardest problems in NP
NP-Complete	Problems that are both in NP and as hard as any problem in NP
Divide and conquer	Split, solve smaller pieces, then combine
Dynamic programming	Store and reuse results from overlapping subproblems
Greedy strategy	Make the best-looking choice at each step
Recursion	A function solves a smaller version of the same problem
Base case	The condition where recursion stops

2



What We Will Use Today

- ▶ Correctness, performance, and scalability as strategy
- ▶ Divide and conquer as a split-and-combine pattern
- ▶ Dynamic programming as recognition of saved repeated work
- ▶ Greedy as local choice with possible risk
- ▶ Iteration and recursion as two strategy forms
- ▶ Comparison table evidence
- ▶ Strategy recommendation with conditions

3

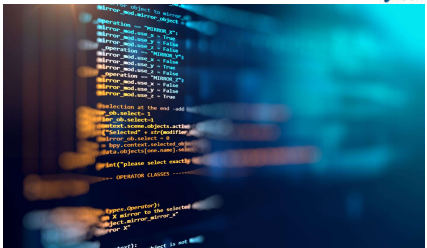


What We Will "Touch On" and Revisit Later

- ▶ Graphs and traversal strategy
- ▶ Recommendation and ranking systems
- ▶ Larger-scale algorithms
- ▶ Explainability and limits of algorithmic solutions
- ▶ Final-assessment explanation and justification



4

Review Coding Lab

5



Review: What Lab 04 Taught Us

Last week, we learned that:

Search strategy depended on assumption

Binary search was powerful only when:

- The data was sorted
- The comparison rule matched the order
- The trace decisions were valid

6

Southwest Tech

Today's Question

How can two correct solutions still be different in quality?

7

Southwest Tech

Same Problem, Different Strategy

Many problems can be solved in more than one way.

Strategies may differ in:

- ▶ how they break down the problem
- ▶ how much work they repeat
- ▶ how easy they are to explain
- ▶ how well they fit the data shape

8

Southwest Tech

One Problem, Two Algorithmic Strategies

Problem: Find the best route

Strategy A: Fewer setup steps, weaker result

1. Quick start
Start with simple route
2. Improve locally
Make small adjustments
3. Get result
Good enough route found

Tradeoff: less analysis, faster, less optimal

Strategy B: More analysis steps, stronger result

1. Understand the map
Collect and review data
2. Explore options
Generate multiple candidate routes
3. Evaluate routes
Analyze costs, risks, and time
4. Choose best
Select the best route
5. Get result
Stronger route found

Tradeoff: more analysis, slower, more optimal

Evidence: compare results

- Which route is better?
- What are the tradeoffs?

Strategy Comparison

Different strategies can produce different tradeoffs. The goal is to choose the strategy that best fits the situation.

9

What Counts as Success Today

- Describe two strategies
- Test both strategies
- Compare correctness and readability
- Discuss likely growth behavior
- Recommend one strategy with conditions

10

Southwest Tech

Textbook Review

Designing Algorithms

The reading focuses on designing algorithms.

Design means choosing a strategy that fits:

- ▶ The required result
- ▶ The problem constraints
- ▶ The data size
- ▶ The cost of the approach
- ▶ The explanation needed

11

Southwest Tech

Textbook Review

Functional Requirements

Functional requirements ask:

- ▶ What must the algorithm produce?
- ▶ What input does it need?
- ▶ What output is expected?
- ▶ What counts as a correct result?

12

Textbook Review 

Non-Functional Requirements

Non-functional requirements ask:

- ▶ How fast should it be?
- ▶ How readable should it be?
- ▶ How much memory can it use?
- ▶ How well should it scale?
- ▶ How explainable does it need to be?



13

Textbook Review 

Three Design Concerns

When designing an algorithm, ask:

1. Is it correct?
2. Is the performance reasonable?
3. Can it scale when the data or demand grows?



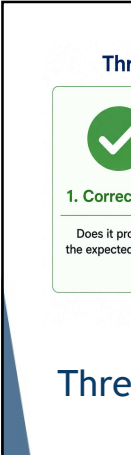
14

Textbook Review 

Three Algorithm Design Concerns

 1. Correctness Does it produce the expected result?	 2. Performance How much work does it take?	 3. Scalability What happens as the data grows?
--	---	---

Three Design Concerns



15

Textbook Review 

Theory Vocabulary

The reading includes skim-level vocabulary:

- ▶ polynomial
- ▶ NP-Hard
- ▶ NP-Complete

For this course:

- ▶ Recognize the terms
- ▶ Understand why some problems are harder
- ▶ Do *not* try to master formal proof



16

Textbook Review 

Strategy Families

The reading introduces strategy families:

- ▶ divide and conquer
- ▶ dynamic programming
- ▶ greedy strategy

It also presents larger examples:

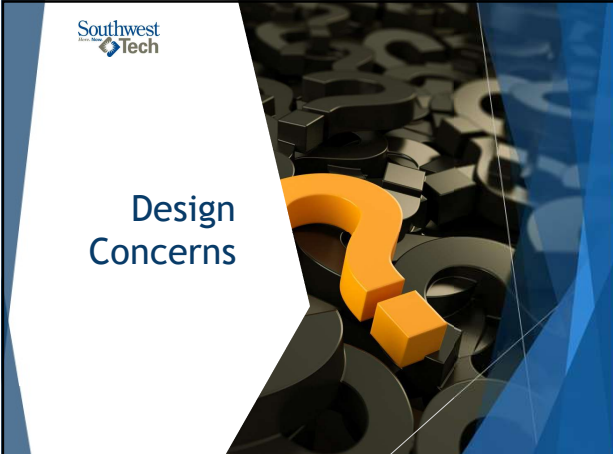
- ▶ PageRank
- ▶ linear programming



17

Southwest Tech

Design Concerns



18

Southwest
Tech

Correctness



Correctness asks:

- ▶ Did the algorithm produce the expected result?
- ▶ Did it handle normal cases?
- ▶ Did it handle edge cases?
- ▶ Can we show evidence?

19

Performance

-  How much work does the strategy do?
-  Does it repeat unnecessary work?
-  How does the work grow?
-  Is this approach reasonable for the problem size?

20

Scalability

-  What happens with larger data?
-  What happens with more users?
-  What happens with frequent updates?
-  Can the approach adapt when demand changes?

21

Strategy Comparison Frame

Criterion	Strategy A	Strategy B
Correctness		
Readability		
Growth		
Fit to data		

22

Southwest
Tech

Better Means Better For This Context



Avoid

- ▶ "This strategy is better."



Use

- ▶ "This strategy is better for this problem because..."
- ▶ - "This strategy may not be better when..."

23

Southwest
Tech

Evidence For Strategy Choice

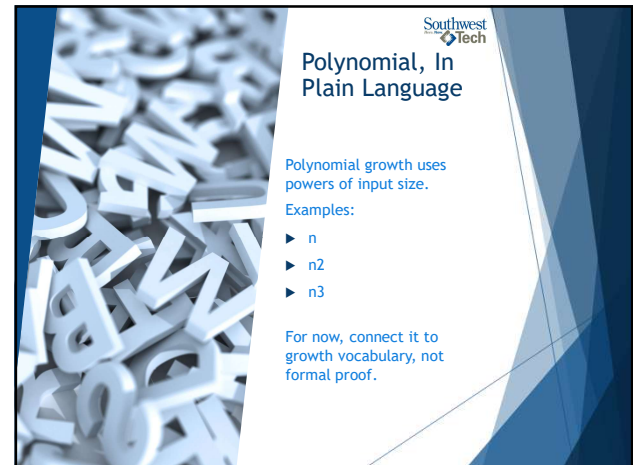
Evidence may include:

- ▶ test cases
- ▶ edge cases
- ▶ trace tables
- ▶ decision trees
- ▶ timing results
- ▶ comparison tables
- ▶ limitation notes

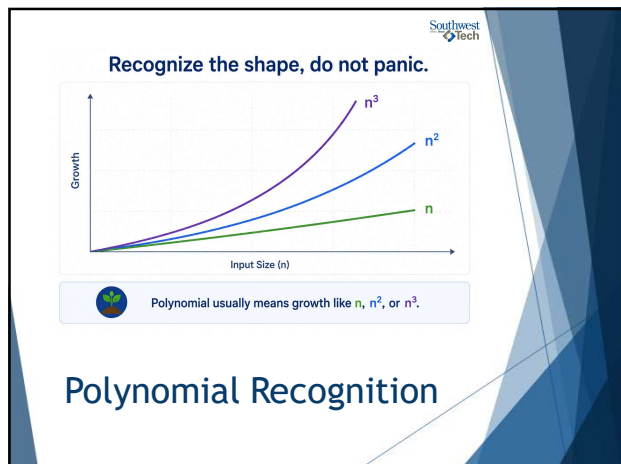
24



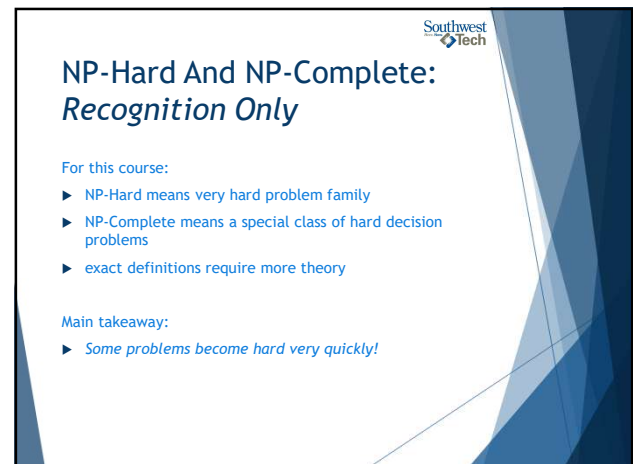
25



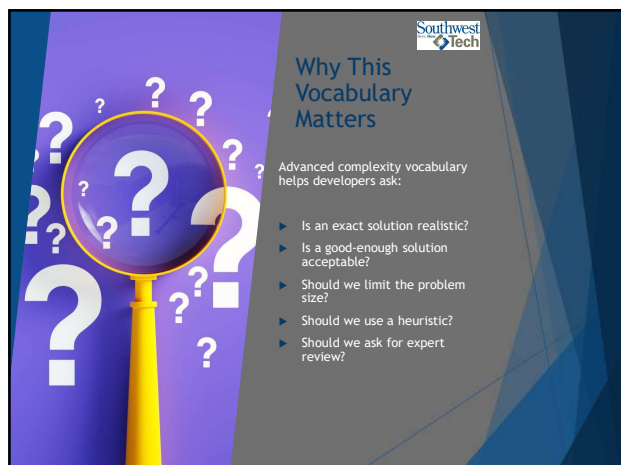
26



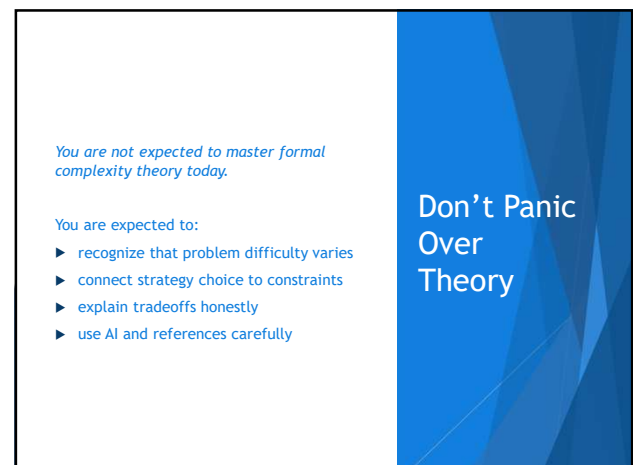
27



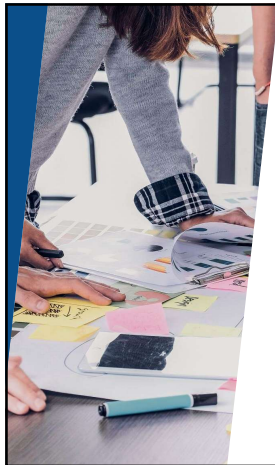
28



29



30



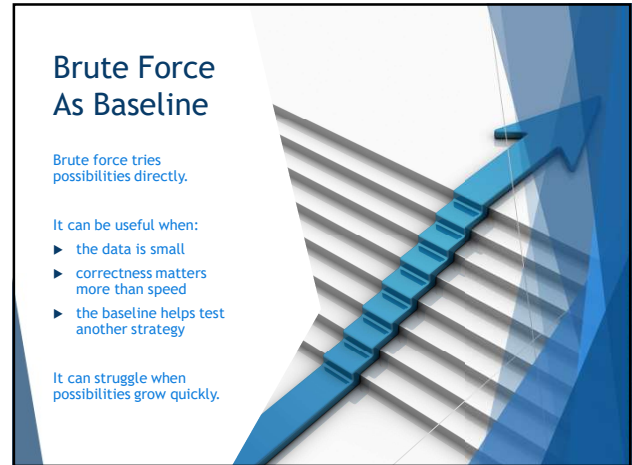
Strategy Is A Design Decision

A strategy decides how the problem will be attacked.

Examples:

- ▶ try everything
- ▶ split the problem
- ▶ reuse earlier results
- ▶ choose the best-looking next step
- ▶ follow a recursive structure

31



Brute Force As Baseline

Brute force tries possibilities directly.

It can be useful when:

- ▶ the data is small
- ▶ correctness matters more than speed
- ▶ the baseline helps test another strategy

It can struggle when possibilities grow quickly.

32



Break

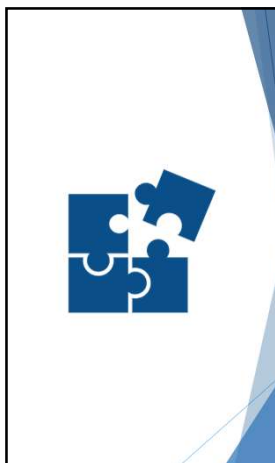
10 Minute "Human" bio-break

33



Algorithmic Strategies

34



Divide And Conquer

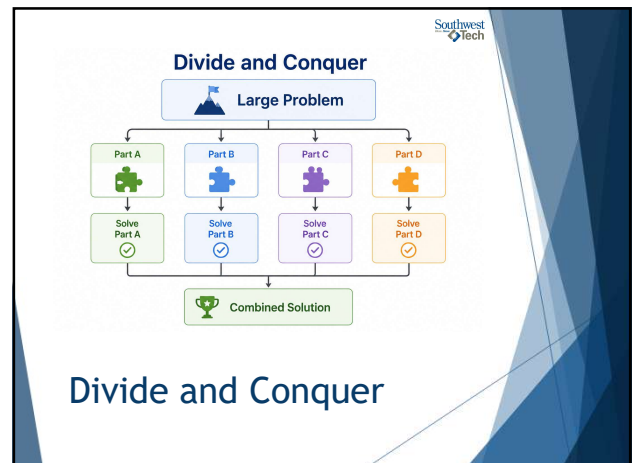
Divide and conquer:

- ▶ divide the problem
- ▶ solve smaller pieces
- ▶ combine the results

Common examples:

- ▶ merge sort
- ▶ binary search
- ▶ some recursive strategies

35



Divide and Conquer

Large Problem

Part A, Part B, Part C, Part D

Solve Part A, Solve Part B, Solve Part C, Solve Part D

Combined Solution

Divide and Conquer

36

Divide And Conquer Fit

Divide and conquer fits when:

- ▶ the problem can be split
- ▶ smaller pieces are similar to the whole
- ▶ results can be combined
- ▶ splitting reduces the work or complexity

37

Dynamic Programming

Dynamic programming is useful when:

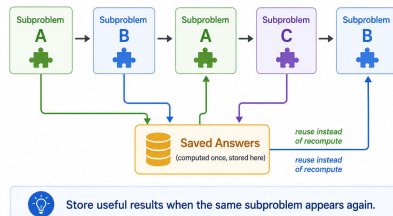
- ▶ the problem has repeated subproblems
- ▶ earlier results can be saved
- ▶ saved results prevent repeated work

For this course: recognize the pattern.

38

Dynamic Programming

Store and reuse repeated subproblem answers.



Dynamic Programming

39

Greedy Strategy

A greedy strategy chooses what looks best right now.

Examples:

- ▶ choose the cheapest item first
- ▶ choose the shortest task first
- ▶ choose the largest coin first
- ▶ choose the highest immediate score

40

Greedy Risk

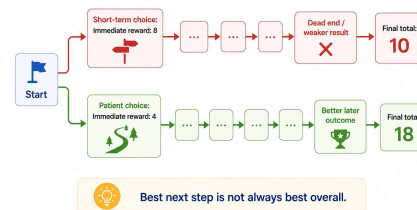
Greedy strategies can fail when:

- ▶ an early choice blocks a better later choice
- ▶ the local score hides a global tradeoff
- ▶ the rule does not match the real goal

Greedy needs evidence

41

The Risk of a Greedy Algorithm



Greedy Risk

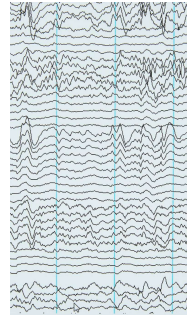
42

Strategy Family Comparison

Strategy	Basic Idea	Watch For
Brute force	try possibilities	too much work
Divide and conquer	split and combine	combine step
Dynamic programming	reuse saved results	identifying repeated subproblems
Greedy	best-looking next step	local choice may fail

43

PageRank As An Example



It considers:

- ▶ links between pages
- ▶ importance passed through relationships
- ▶ repeated updating of scores
- ▶ ranking based on structure

44

PageRank: What To Notice

Notice the algorithmic questions:

- ▶ What is represented?
- ▶ What is repeated?
- ▶ What score changes over time?
- ▶ What does the rank claim?
- ▶ What are the limits of that claim?

45

Linear Programming: Recognition

Linear programming helps optimize under constraints.

Plain-language frame:

- choose values
- follow constraints
- maximize or minimize a goal

For this course: recognize the idea.

46

Constraints Shape Strategy

The same problem can change strategy when constraints change.

Examples:

- ▶ exact answer required
- ▶ good-enough answer acceptable
- ▶ small data
- ▶ large data
- ▶ limited memory
- ▶ explanation required

47

Strategy Before Code

Before coding, describe:

- ▶ Strategy A
- ▶ Strategy B
- ▶ what each strategy assumes
- ▶ what evidence will compare them
- ▶ what would make one a better fit



48

Southwest Tech

Same Strategy, Different Form

Sometimes the strategy changes.

Sometimes the representation changes.

Sometimes both solutions solve the same task, but one fits the data shape more clearly.



49

Southwest Tech

Iteration and Recursion



50

Southwest Tech

Iteration

Iteration repeats steps using a loop.

Common signs:

- ▶ `for`
- ▶ `while`
- ▶ running total
- ▶ index or counter
- ▶ explicit work list or stack



51

Southwest Tech

Recursion

Recursion happens when a function solves a smaller version of the same problem.

It needs:

- ▶ a base case
- ▶ a recursive step
- ▶ progress toward stopping



52

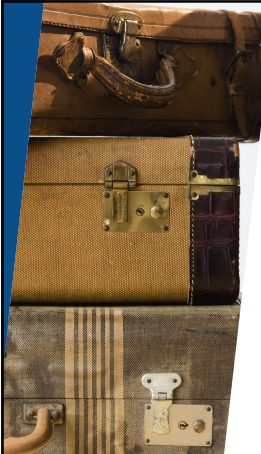
Southwest Tech

Base Case

The base case answers:

- ▶ When are we done?
- ▶ What result can we return directly?
- ▶ How do we stop the repeated calls?

No base case means no reliable recursion.



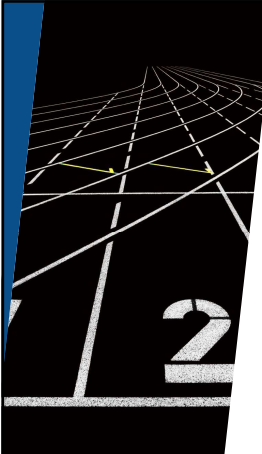
53

Southwest Tech

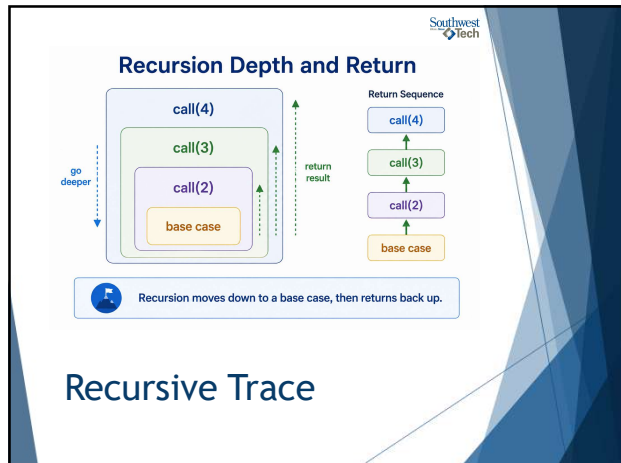
Recursive Trace

A recursive trace can show:

- ▶ call depth
- ▶ current input
- ▶ action taken
- ▶ returned result
- ▶ how totals combine



54



55

Iteration vs Recursion

Question	Iteration	Recursion
How does it repeat?	loop	function calls itself
What must be tracked?	loop state	call state and base case
When does it fit?	flat repeated work	nested or self-similar work

56



57

Demo Scenario

Problem:

- Compare two strategies:
 - iterative processing with an explicit work stack
 - recursive processing of nested groups

Goal

Calculate the total value of nested donation envelopes

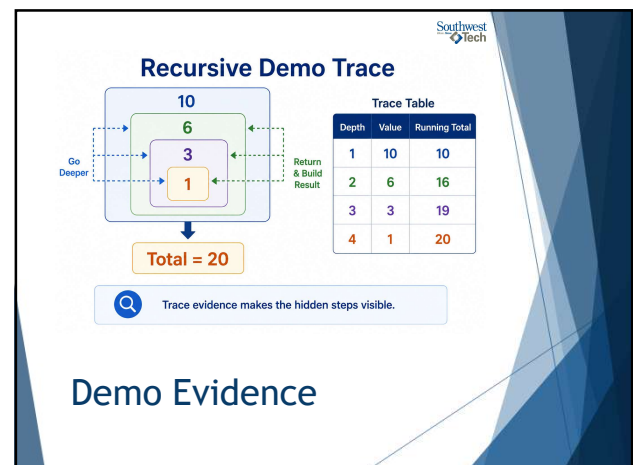
58

Southwest Tech

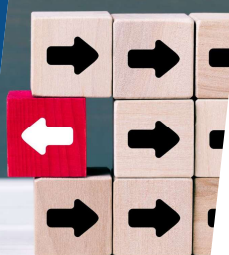
Demo Evidence

- Iterative total
- Recursive total
- Recursive call trace
- Strategy comparison table
- Key takeaway

59



60



Demo Comparison

Compare:

- ▶ correctness
- ▶ readability
- ▶ growth
- ▶ fit to data

The answer alone is not the whole evaluation.

61



Demo Explanation

"Both strategies produce the same total. The recursive strategy fits the nested data more directly because each nested group can be treated as a smaller version of the same problem."

62



From Demo to Lab

In Lab 05 you will compare two strategies for a different problem.

Possible options include:

- ▶ cumulative product
- ▶ grouped values
- ▶ decision tree
- ▶ coin-change or greedy selection
- ▶ scheduling or shopping tradeoffs

63

Lab 05 Evidence

problem statement	two strategy descriptions	two implementations or simulations
at least four tests	trace, decision tree, or comparison table	recommendation and limitation

64

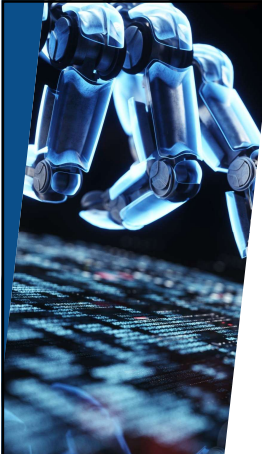


README Evidence

Your README should show:

- ▶ what problem you solved
- ▶ what two strategies you compared
- ▶ what tests you ran
- ▶ what evidence you collected
- ▶ which strategy you recommend
- ▶ when your recommendation might change
- ▶ AI-use note, if applicable

65



AI Use In Lab 0n

Use AI after you have:

- ▶ framed the problem
- ▶ described your first strategy
- ▶ attempted or planned evidence

AI may help:

- ▶ suggest a second strategy
- ▶ critique tradeoffs
- ▶ debug one implementation

66



67

What To Carry Forward

Strategy choice is contextual.
Ask:

- ▶ Does it produce the expected result?
- ▶ Is it readable enough to explain?
- ▶ How does the work grow?
- ▶ Does it fit the data shape?
- ▶ What tradeoff did I accept?

68

How To Use The Textbook For Next Week's Reading

Focus on:

- ▶ graph basics: nodes, edges, links, and networks
- ▶ graph representation
- ▶ simple, directed, undirected, and weighted graphs
- ▶ ego networks and neighborhoods
- ▶ shortest path intuition
- ▶ BFS and DFS traversal

Optional reading includes centrality, density, triangles, graph visualization, and social network analysis

69

Thank you!

Have Fun!

70